A1 学生版讲义 · 开场与需求 PRD

> **课程**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元① **用法**:课前预习、课后复习都能看。内容精炼,配合课堂消化。 **今天不写业务代码**,今天写的是"给 AI 的图纸"。

一、Vibe Coding 范式:你的角色变了

这门实训全程用 **Vibe Coding**:你不再一行行敲 Java、拼 Socket、写 JDBC,而是**指挥 AI(Trae SOLO)来写**,你负责**想清楚 + 表达 + 判断 + 审查**。

三句话口诀:**人定规范 · AI 写码 · 人审查**。

- **AI 写代码,你练思维**:把要做的事想清楚、说清楚,再看 AI 做得对不对。Socket 收发、JDBC 样板 AI 闭着眼睛写,你的价值是**判断它写得对不对**(比如借书有没有放进事务、防没防超借)。
- **慢在哪变了**:以前慢在敲字、查 API;现在 AI 生成很快,慢在"你有没有想清楚"。
- **你的核心竞争力**:表达需求 + 审查结果。今天开始,你从"写代码的人"升级成"**指挥 + 审查 AI 的人**"。

二、这五个单元做什么(Part A · 老师带做)

用 Trae 从零搭一套能局域网多机演示的 C/S 图书馆:

```
单元① 需求 + Agent PRD   ← 今天:给 AI 图纸,让 Trae 存成 .trae/rules
单元② 生成服务端          建库 + Socket 多线程服务端 + 打通登录链路
单元③ 生成 Swing 客户端   读者端查书/借书/还书 + 联调 + 代码审查
单元④ 迭代               管理员端 + 搜索分页 + 服务端监控窗口
单元⑤ 上线               打包双 jar + 局域网多机演示 + 复盘(+ 可选 AI 助手)
```

Part A 老师带做;之后 **Part B 分组**,各组选一个项目用同一套方法独立做、答辩验收。这一切的起点,是今天写好的这份需求图纸。

三、Trae SOLO:你说人话,它写代码并存文件

Trae 是一个带 AI 助手的编程 IDE。**本课只用它的 SOLO 模式**,一个模式够我们做完整个 C/S 图书馆。

- 打开 Trae,切到 **SOLO** 模式。
- 在对话框用**大白话**说要做什么,回车。
- AI 会:理解需求 → 自动建项目/写代码/**保存文件** → 给你说明或让你运行。
- **不满意就继续对话**:"把 XX 改成 YY,别的不动。"

> 记住:SOLO 模式 = 你说人话,它写代码**并存文件**。**因为它能自己创建、保存文件,所以后面写 PRD 你不用手抄——一句话让它存就行。**(换成 Claude Code 也一样,`.trae/rules` 对应它的 `CLAUDE.md`。)

环境提醒:C/S 项目要用 JDK 11 + MySQL 8(`127.0.0.1:3306` / root / 12345678),课前装好,别上课卡在装环境。

三·补、🔥 上手热身:城市探索小页面(只今天做一次)

学会 SOLO 怎么用后,先别急着上 C/S 图书馆——**这是第一次用 Trae 的上手热身**:用它做个**纯前端网页**(城市探索指南:渐变首屏 + 搜索、分类卡片、推荐卡片、点卡片弹详情)尝个甜头,不碰后端、不用框架、不接数据库,几分钟出活。**热身只今天做一次,做完直接上我们的桌面 C/S 图书馆(Swing + Socket + JDBC)。** 对标同目录《城市探索-热身案例-答案版.html》。

四步 prompt 速查(一步一句、"只做 XX 不动其他"):

```
① 帮我做一个"城市探索指南"的纯前端小网页(只用 HTML+CSS+JS,不要后端、不要框架)。先做整体:一个渐变紫色的首屏,标题"城市探索指南",下面一个圆角搜索框。
② 基于现有页面,在首屏下面加一排分类卡片:美食🍜、风景🏞️、咖啡☕、文艺🎨、购物🛍️,每张显示 emoji + 分类名 + 数量,鼠标悬停有上浮效果。只加分类,别动首屏。
③ 基于现有页面,在分类下面加"本周热门推荐",卡片网格展示 6 个地点:每张有渐变色封面(色块代替图片)+ 地点名 + 分类标签 + 一句描述 + 评分。数据你先编 6 条合理的。只加推荐区。
④ 基于现有页面,加两个交互:① 点推荐卡片弹详情弹窗(封面/名字/标签/描述 + 关闭按钮);② 搜索框输入关键词回车,按地点名或分类过滤推荐列表。只加这两个交互,别改布局。
```

- 体会:随口说,AI 就做出能看能点的页面(Vibe Coding 的爽点)。
- 也埋个伏笔:AI 编的数据/配色和你想的不一定一样——热身是网页几步就成,而真项目是要局域网多机、防并发超借的桌面 C/S 图书馆,要做准做全,就需要下面的"图纸"(Agent PRD)。

四、好需求的三标准

给 AI 的需求,用这三把尺子量一量。

标准	反例 ❌	正例 ✅
具体	"做个好看的图书列表"	"JTable 展示书名/作者/分类/库存四列,双击一行弹详情"
有边界	"做个图书系统"	"先只做:登录 + 查询 + 借还书 + 管理员 CRUD,罚款/AI 助手以后再说"
可验证	"让它快一点"	"发 `LOGIN|admin|123456`,服务端返回 `OK|admin|系统管理员|1`"

> 模糊的需求 = 模糊的结果。C/S 项目尤其要能验证——链路通没通,一发一收就知道。

五、⭐ 规范是新的源代码

本课最核心的一句话(来自斯坦福 CS146S):

> **在 AI 时代,你写的"需求 / 规范"比"代码"更重要。代码只是规范的"有损投影"。**

打个比方:

- **规范 = 图纸,代码 = 施工**。你的活是**画好图纸**;施工(写代码)交给 AI。
- 图纸好,AI 可以**反复施工**(重新生成、改需求再生成)都不走样;图纸烂,AI 再强也盖不出好房子。

> 过去练的是"Socket 怎么写、事务怎么开",以后练的是"图纸怎么画"。你写的那份需求文档,就是新时代的源代码。

六、上下文的四种失败模式

AI 做不对,先别怪它笨——先检查是不是你给的上下文出了这四种问题。

失败模式	什么意思	C/S 图书馆里的例子
🔴 **不足**	没告诉 AI 背景,它瞎猜	没说"客户端不直连库、经服务端",AI 在界面里直接写 JDBC
🔴 **过载**	一次塞太多,抓不住重点	一句话把服务端+五窗体+借书+统计全说了,又杂又乱还漏并发
🔴 **矛盾**	不同地方说法打架	一会儿端口 9000,客户端又连 8080;一会儿 Swing 一会儿冒出 Web
🔴 **过期**	项目变了规范没更新	协议加了新命令但 Rules 没改,AI 按旧协议生成

> 解药就是下面的 Agent PRD:把规范固定下来、写清楚、及时更新。

七、大任务拆解 + 纠错三步法

用 AI 干活的两个基本功,后面每一步都在用。

① **大任务拆解**:别指望一句话让 AI 把整个系统做出来(那就是"上下文过载")。**要拆成小步**,一次只让 AI 做一块——"只做 XX,不动其他"。

> 做 C/S 图书馆 → 建库 → 服务端骨架 → 打通登录链路 → 登录窗 → 查书 → 借书 → 还书 → 管理员端 → 监控窗口 → 打包。 🔑 **C/S 心法**:先把"客户端 → 服务端 → 数据库 → 返回"**一条链路(登录)打通**,后面所有功能都是复制它换业务。**别先堆界面**。

② **纠错三步法**(AI 做错了怎么办):

1. **定位**:哪里不对——哪个功能、什么现象。
2. **描述**:说清"发 X 应该 Y,但实际是 Z"。
3. **修复**:让 AI **只改那一处**——"只把这里从 A 改成 B,别的不动"。

> 口诀:**先改需求,再让 AI 改代码。** 别自己上手删代码,越删越乱——把问题描述清楚交给 AI。

八、⭐ C/S 架构两档:图书馆为什么走档 B

C/S = 客户端-服务器。核心问题:**客户端(Swing 桌面程序)怎么和数据库打交道?** 分两档——这是**你(人)定规范**的关键,选错档 AI 会顺着错架构一路生成。

	档 A 两层	档 B 三段式(我们)
结构	`客户端 ──JDBC──> 库`	`客户端 ──Socket──> 服务端 ──JDBC──> 库`
数据库密码	每个客户端都有(不安全)	只在服务端(安全)
事务把关	散在各客户端	服务端集中把关
多人联网	别扭	天然支持(局域网多机)
适合	单机小管理系统	多客户端、要并发一致
技术点	Swing + JDBC	Swing + **Socket + 多线程** + JDBC + 事务

图书馆走档 B,因为要满足两条硬需求:**局域网多台客户端 + 借书并发防超借**。

- 客户端不直连库,只发一行文本(如 `LOGIN|admin|123456`),服务端查库处理再返回一行(`OK|..` / `ERR|..`)。
- 服务端**多线程**:每来一个客户端,`accept()` 后开一个线程去服务——**每客户端一线程**,多读者同时在线互不阻塞。

> 💬 打比方:档 A 是"每个员工都有仓库钥匙自己去拿货";档 B 是"员工跟前台(服务端)说要什么,前台统一去拿、统一记账,钥匙只在前台"。 🔭 Part B 分组:管理类(超市/酒店/停车场)可走档 A;通信类(聊天室/对战)走档 B 推送型;图书馆是最经典的档 B 请求-响应型。

九、Agent PRD = 给 AI 的图纸(八块)

Agent PRD 不是写给人看的文档,是**写给 AI 看的项目说明**,把项目的规范全固定下来。C/S 项目核心八块:

#	块	内容要点
1	**项目概述**	校园图书馆 C/S 桌面程序,管理员 + 读者。
2	**技术选型/架构**	Java Swing + Socket + JDBC + MySQL;**档 B 三段式,客户端不直连库**;端口 9000。
3	**数据模型**	t_user / t_category / t_book / t_borrow 四张表 + 字段 + 主外键。
4	**通信协议**	一行 `命令|参数,OK|数据` / `ERR|原因`;LOGIN/BOOK_QUERY/BORROW/RETURN…
5	**业务规则**	角色权限、借书事务(减库存 + 写记录同生共死)、防超借、借阅上限、密码底线。
6	**并发与多线程**	服务端**每客户端一线程**;借书并发靠事务 + `WHERE stock>0` 影响行数。
7	**编码规范**	分层、SQL 不写界面里、PreparedStatement 防注入、UTF-8、中文注释。
8	**界面清单**	≥10 个窗体,登录窗含服务器地址;主色蓝,JTable 列表。

> **Trae 的 Rules 文件(`.trae/rules`)就是 Agent PRD 的载体**:写好放进项目,Trae 每次生成代码都会自动读它、照它来。**C/S 项目尤其要把"通信协议 + 每客户端一线程 + 借书事务"写死**,否则 AI 容易漏并发、或把 SQL 塞进界面。

十、两段 prompt 速查(可复制)

① 反向提问:让 AI 帮你把需求问清楚

新手最大的问题是"想不全"。绝招:**让 AI 反过来问你**。把下面这段贴进 Trae SOLO:

```
我要做一个校园图书馆借阅系统,是客户端-服务器的桌面程序,但需求没完全想清楚。
请你像产品经理一样,每次只问我一个问题,一共问 6-8 个,帮我把需求理清楚;我回答完你再问下一个。
```

AI 会一个一个问(谁用/角色?图书借阅记哪些字段?借书规则?**技术架构**?**通信协议**?**并发怎么办**?界面几个窗口?),你依次回答——**这些答案就是需求**。把"想不清楚"变成"一问一答"。

② 让 Trae 把需求整理成规范并保存(不用手抄)

需求问清楚后,**别自己手动整理粘贴**——直接让 Trae 干,并让它存进文件:

```
把我们刚理清的需求,整理成一份项目规范,保存到项目根目录的 .trae/rules 文件。要包含:
① 项目概述 ② 技术选型(Java Swing + Socket + JDBC + MySQL,档B三段式C/S,客户端不直连库;端口 9000)
③ 数据模型(t_user/t_category/t_book/t_borrow 四表 + 字段 + 主外键)
④ 通信协议(一行 命令|参数,OK|数据 / ERR|原因;含 LOGIN/BOOK_QUERY/BORROW/RETURN 等)
⑤ 业务规则(角色权限;借书同一事务减库存+写记录、防超借 WHERE stock>0 判影响行数、借阅上限)
⑥ 并发多线程(服务端每客户端一线程)⑦ 编码规范(分层、PreparedStatement、UTF-8)
⑧ 界面清单(≥10 窗体,登录窗含服务器地址+端口)。
```

> Trae 会**自己创建 `.trae/rules` 文件并写入**——你一个字没抄。**存好后一定要打开 `.trae/rules` 对照标准审查**(架构是不是档 B、通信协议在不在、借书事务/防超借/每客户端一线程有没有),缺的用纠错三步法让它补。

十一、FAQ

Q1:全程让 AI 写,我岂不是什么都不会? 恰恰相反。你练的是更值钱的能力:想清楚需求、判断 AI 写得对不对。你知道 C/S 图书馆的命根子是"协议 + 每客户端一线程 + 借书事务",AI 漏了你一眼看出来——这双"判断 AI 的眼睛",就是你的核心竞争力。

Q2:AI 做出来的和我想的不一样,怎么办? 先别怪 AI。第一步对照"上下文四种失败模式"自查:是不是背景说得不足?一次塞太多?前后说法矛盾?规范过期了?然后用纠错三步法——**先改需求(把规范补对),再让 AI 改代码**:"发 X 应该 Y,但实际是 Z,只把这里从 A 改成 B,别的不动。"

Q3:为什么先写 PRD,不直接让 AI 写代码? 因为"规范是新的源代码"。图纸(PRD)画清楚,AI 后面每一步生成都照图施工、不走样;直接让它写,它可能把客户端写成直连库(档 A)、漏掉多线程,做出四不像。前期理清需求,省掉后面反复返工。

Q4:PRD 要不要我一字一句手抄进文件? 不用。你只负责"想清楚 + 说清楚",然后一句话让 Trae 把问答整理成规范、自己存进 `.trae/rules`(见速查 prompt ②)。**你的活是审查**:打开它对照标准看对不对,缺的让它补。让 AI 干活、你把关——这就是"信任但验证"。

Q5:档 A 和档 B 到底怎么选? 要多人同时联网、要并发一致(防超借)→ 走**档 B 三段式**;单机或一两人用的小工具 → 走**档 A 两层**。我们的图书馆要局域网多机 + 防并发超借,所以走档 B。选档是"人定规范"的一部分,写进 PRD 技术选型里。

Q6:C/S 项目里 AI 最容易漏什么? 三条:① 把客户端写成直接连数据库(退回档 A);② 忘了通信协议;③ 借书没放进事务、没防超借,或服务端没做到每客户端一线程。审查时重点盯这三条。

今日产出

用**反向提问法**把 C/S 图书馆需求理清楚,**让 Trae 把它整理成 Agent PRD 并存进 `.trae/rules`**,再对照抓手1 标准审查补齐**架构档 B + 通信协议 + 借书事务/防超借/每客户端一线程——给 AI 的图纸备好了**,后面四个单元的生成质量都靠它。